

Comparing Novel Thermodynamic Computing with Traditional Graphics Processing Unit Implementations for Ground State Energy Estimation in Classical Optimization Problems

Noah Kestenholz

Department of Advanced Computing Sciences

Faculty of Science and Engineering

Maastricht University

Maastricht, The Netherlands

Abstract—To be filled in.

Index Terms—thermodynamic computing, THRML, Ising model, MaxCut, Gibbs sampling, Block Gibbs, Metropolis-Hastings, probabilistic computing, energy-based models, EBMs

I. INTRODUCTION

Solving hard combinatorial optimisation problems is central to applications spanning logistics, drug discovery, and machine learning, yet the energy cost of running these workloads on conventional hardware continues to grow. Thermodynamic computing has emerged as a promising alternative paradigm: rather than suppressing thermal noise, it harnesses it as a computational resource [7], [8].

The Ising model is the canonical test bed for combinatorial solvers. It assigns a binary spin $s_i \in \{-1, +1\}$ to each node of a graph, with an energy function whose minimisation is NP-hard in general. Any NP problem can be mapped to an Ising instance with only polynomial overhead [1], making progress on Ising solvers broadly significant. This thesis studies two regimes: the *ferromagnetic* case ($J_{ij} > 0$), where aligned spins are favoured and the energy landscape is relatively smooth, and the *anti-ferromagnetic* case ($J_{ij} < 0$), which produces frustrated, rugged landscapes. The MaxCut problem — partition a graph to maximise the number of crossing edges — maps exactly to anti-ferromagnetic Ising and serves as the natural stress test for any sampler [11].

The standard baseline for ground state estimation is Metropolis-Hastings (MH), whose rejection step allows the Markov chain to escape local minima and guarantees convergence to the target distribution. GPU implementations exploit the local neighbourhood structure of spin models to achieve speed-ups of up to three orders of magnitude over serial CPU code [2], [3]. Gibbs sampling eliminates rejection by sampling each spin directly from its conditional distribution, enabling

whole colour classes of conditionally independent spins to be updated simultaneously. However, the absence of a rejection step means the sampler cannot cross low-probability barriers — a failure mode that is particularly severe in frustrated anti-ferromagnetic systems [4]. *Block* Gibbs sampling updates conditionally independent groups in parallel, gaining throughput over single-site Gibbs while retaining this fundamental limitation.

Extropic’s Thermodynamic Sampling Unit (TSU) is built around block Gibbs, deliberately forgoing rejection in exchange for the energy efficiency that probabilistic hardware enables. Jelinčič et al. project performance parity with GPUs at roughly 10,000 times lower energy [7], building on earlier work on p-bit circuits [5] and sparse Ising Machines [6]. The THRML library [9] simulates TSU-style block Gibbs on a conventional GPU, enabling algorithmic study before physical hardware exists at scale.

Despite these advances, the empirical performance trade-offs of block Gibbs across different problem classes remain uncharacterised. It is unclear whether THRML block Gibbs achieves solution quality comparable to MH, and under which conditions its computational efficiency advantage holds. This thesis addresses that gap through a systematic three-way comparison of THRML block Gibbs, standard single-site Gibbs, and Metropolis-Hastings, all running on identical GPU hardware. Experiments cover three graph topologies — 2D grid, random regular graph, and Erdős–Rényi — across ferromagnetic Ising, anti-ferromagnetic Ising, and MaxCut instances, with system sizes up to [TBD] nodes and a sweep over inverse temperature β . Three research questions guide the work:

- 1) How do the three samplers compare in solution quality across problem types and system sizes?
- 2) How does their relative performance scale with graph topology and temperature?
- 3) What are the computational efficiency trade-offs, as measured by effective samples per second and Gelman–Rubin convergence diagnostics?

The remainder of this thesis is structured as follows: Section II describes the three sampler implementations; Section III defines the experimental methodology; Section IV presents results; Section V interprets findings in the context of existing literature; and Section VI draws conclusions.

II. METHODS

A. Problem Formulation

All three samplers operate on the same shared graph representation. An Ising model instance is defined by a graph $G = (V, E)$ with $n = |V|$ nodes, edge-wise coupling strengths $J_{ij} \in \mathbb{R}$, and inverse temperature $\beta > 0$. Each node i carries a spin $s_i \in \{-1, +1\}$, and the energy of a configuration $\mathbf{s}$ is:

$$\mathcal{E}(\mathbf{s}) = -\beta \sum_{(i,j) \in E} J_{ij} s_i s_j \quad (1)$$

The samplers target the Boltzmann distribution $P(\mathbf{s}) \propto \exp(-\mathcal{E}(\mathbf{s}))$. For ferromagnetic instances $J_{ij} = +1$, aligned spins are energetically favoured; for anti-ferromagnetic instances $J_{ij} = -1$, opposing spins are favoured. MaxCut is handled through the same formulation by setting $J_{ij} = -1$, so that minimising (1) is equivalent to maximising the number of cut edges. The cut value is computed separately as $\text{Cut}(\mathbf{s}) = \sum_{(i,j) \in E} |J_{ij}| (1 - s_i s_j)/2$, which for unit weights ($|J_{ij}| = 1$) counts the number of cut edges.

B. Metropolis-Hastings

At each step, a node i is chosen uniformly at random and a spin flip is proposed. The energy change from flipping s_i depends only on the local neighbourhood $\mathcal{N}(i)$:

$$\Delta\mathcal{E} = 2\beta s_i \sum_{j \in \mathcal{N}(i)} J_{ij} s_j \quad (2)$$

The flip is accepted with probability $\min(1, \exp(-\Delta\mathcal{E}))$. One sweep consists of n such steps. The acceptance step guarantees detailed balance and allows the chain to cross low-probability barriers, at the cost of rejected proposals that leave the state unchanged.

The implementation uses a padded neighbour list of shape $[n, d_{\max}]$, where $d_{\max}$ is the maximum node degree, with a boolean mask to handle varying degree. The inner loop over n steps is compiled with `jax.lax.scan`; PRNG keys are derived on-the-fly via `jax.random.fold_in` to avoid materialising large key arrays in GPU memory. The sampler is JIT-compiled with `jax.jit` and parallelised over independent chains with `jax.vmap` [10].

C. Standard Gibbs Sampling

Gibbs sampling draws each spin exactly from its conditional distribution given all neighbours, eliminating the accept/reject step entirely. For node i :

$$P(s_i = +1 \mid \mathbf{s}_{\setminus i}) = \sigma \left(2\beta \sum_{j \in \mathcal{N}(i)} J_{ij} s_j \right) \quad (3)$$

where σ is the sigmoid function. One sweep consists of n single-node conditional updates. The implementation shares the same neighbour list structure and `lax.scan/vmap` pattern as MH; the only difference is replacing the Metropolis accept/reject with direct conditional sampling from (3).

D. Block Gibbs via THRML

Block Gibbs extends single-site Gibbs by updating entire groups of conditionally independent nodes in parallel. Two nodes are conditionally independent given all others if and only if they share no edge, so the natural grouping is a proper graph colouring: nodes of the same colour can be sampled simultaneously without violating the Markov property.

The graph is coloured using the DSATUR greedy algorithm, producing colour classes $\mathcal{C}_1, \dots, \mathcal{C}_k$. One block Gibbs step cycles through all classes in order; within each class, all spins are sampled in parallel from their conditionals (3). A 2D square grid is bipartite and requires two colours; a degree-3 random regular graph typically requires three or four. More colours mean smaller parallel blocks but the same total work per step.

This sampler is implemented using the THRML library [9], which provides an `IsingEBM` energy model, an `IsingSamplingProgram` encoding the colour-block structure, and a `sample_states` function that executes the schedule. The same `jax.vmap` parallelism over independent chains is applied externally.

E. Shared Infrastructure and Timing

All three samplers share a common warm-up phase of n_{warmup} sweeps that are discarded before sample collection. Samples are then collected every n_{thinning} sweeps to reduce autocorrelation. Wall-clock time is measured using `jax.block_until_ready` to ensure all GPU computation is complete before the timer stops, and a separate untimed compilation run precedes each timed run to exclude JIT overhead. This ensures wall-clock times reflect steady-state throughput and are directly comparable across samplers.

III. EXPERIMENTS

The experiments are designed to answer the three research questions by varying problem type, graph topology, system size, and temperature independently, while holding all sampler hyperparameters fixed across conditions. This section describes the experimental design, the configurations tested, and the metrics collected.

A. Experimental Design

Each experiment runs all three samplers — MH, standard Gibbs, and THRML block Gibbs — on the same problem instance with independently sampled random initialisations, chain count, and sample budget. This ensures differences in outcome reflect algorithmic behaviour rather than initialisation or resource differences.

All experiments were run on a single laptop NVIDIA RTX 3060 (6 GB VRAM). Each sampler was given $n_{\text{chains}} = 50$

independent parallel chains, initialised from random spin configurations. Each chain ran $n_{\text{warmup}} = 200$ warm-up sweeps (discarded), then collected $n_{\text{samples}} = 500$ samples with $n_{\text{thinning}} = 5$ sweeps between each sample. JIT compilation time was excluded from all timing measurements.

B. Problem Instances and Configurations

Four groups of configurations were benchmarked, covering the three problem types and three graph topologies.

1) *Ferromagnetic and Anti-ferromagnetic Grid*: A 2D square lattice with $J = +1$ (ferromagnetic) and $J = -1$ (anti-ferromagnetic) at $\beta = 1.0$. System sizes of 10×10 , 20×20 , and 30×30 ([TBD up to $\approx 100 \times 100$]) are tested to measure scaling behaviour. The 2D grid is bipartite, so THRML requires only two colour classes regardless of size.

2) *Temperature Sweep*: A ferromagnetic grid ($J = +1$) at inverse temperatures $\beta \in \{0.1, 0.5, 1.0, 2.0\}$, sweeping from the high-temperature disordered phase ($\beta = 0.1$) to the strongly ordered phase ($\beta = 2.0$). The system size is [TBD, targeting $\approx 50 \times 50$] — larger than the 10×10 used in preliminary runs, as small systems converge easily at all temperatures and obscure sampler differences. A larger grid makes the ordered phase genuinely hard to mix on, revealing whether THRML's throughput advantage holds across the full temperature range rather than at a single operating point.

3) *Random Regular Graph*: A random regular graph of degree d with ferromagnetic coupling $J = +1$ at $\beta = 1.0$. Node counts of 100, 400, and 900 ([TBD]) are tested at degree $d = 3$, with an additional sweep over degrees $d \in \{5, 50?, 100?\}$ ([To clarify]) at fixed node count to isolate the effect of graph density on block Gibbs parallelism. Higher degree increases the chromatic number and reduces the size of each parallel colour block.

4) *MaxCut*: An Erdős–Rényi graph with edge probability $p = 0.1$ and $J = -1$ at $\beta = 1.0$. Node counts of 100, 400, and 900 ([TBD up to GPU memory limit]) are tested. MaxCut on dense non-bipartite graphs is the hardest regime for Gibbs-based samplers, as frustrated anti-ferromagnetic interactions create many competing local minima between which the sampler cannot cross without rejection.

C. Metrics

Four metrics are recorded for each configuration and sampler:

- **Solution quality**: the mean best energy (or cut value for MaxCut) found across chains, and the distribution of per-chain best values. Statistical significance of differences between samplers is assessed using the Mann-Whitney U test on the 50 per-chain best values, with effect size reported as the rank-biserial correlation r .
- **Convergence**: the Gelman-Rubin $\hat{R}$ statistic [12] computed on the energy trajectory across all 50 chains. Values $\hat{R} < 1.1$ indicate convergence; values substantially above 1.1 indicate chains have not mixed.
- **Sampling efficiency**: the effective sample size (ESS) per chain, estimated via integrated autocorrelation time using

the FFT, and ESS per second as the primary throughput metric.

- **Wall-clock time and memory**: total wall time for the timed run (excluding JIT compilation), and peak GPU memory usage recorded via `jax.devices()[0].memory_stats()`.

IV. RESULTS

V. DISCUSSION

VI. CONCLUSION

REFERENCES

- [1] N. Mohseni, P. L. McMahon and T. Byrnes, “Ising machines as hardware solvers of combinatorial optimization problems,” arXiv:2204.00276, 2022.
- [2] M. Weigel, “Performance potential for simulating spin models on GPU,” *J. Comput. Phys.*, arXiv:1101.1427, 2012.
- [3] M. Weigel and T. Yavors’kii, “GPU accelerated Monte Carlo simulations of lattice spin models,” arXiv:1107.5463, 2011.
- [4] C. Roussel, S. Cocco and R. Monasson, “Barriers and dynamical paths in alternating Gibbs sampling of Restricted Boltzmann Machines,” arXiv:2107.06013, 2021.
- [5] K. Y. Camsari, B. M. Sutton, and S. Datta, “p-Bits for probabilistic spin logic,” arXiv:1809.04028, 2019.
- [6] K. Y. Camsari et al., “Massively parallel probabilistic computing with Sparse Ising Machines,” arXiv:2110.02481, 2021.
- [7] V. Jelinčič et al., “An efficient probabilistic hardware architecture for diffusion-like models,” arXiv:2510.23972, 2025.
- [8] D. Melanson et al., “Thermodynamic computing system for AI applications,” *Nat. Commun.*, 2025. doi: 10.1038/s41467-025-59011-x.
- [9] Extropic, “THRML: Thermodynamic Hypergraphical Model Library,” 2025. [Online]. Available: <https://docs.thrml.ai>
- [10] J. Bradbury et al., “JAX: composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <http://github.com/google/jax>
- [11] M. X. Goemans and D. P. Williamson, “Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming,” *J. ACM*, vol. 42, no. 6, pp. 1115–1145, 1995.
- [12] A. Gelman and D. B. Rubin, “Inference from iterative simulation using multiple sequences,” *Statist. Sci.*, vol. 7, no. 4, pp. 457–472, 1992.